

Assignment: Build "DishBot" – An Agentic Support System for DishTV

Tech Stack: Python, LangGraph, LangChain, FastAPI, MongoDB, Pydantic, LangSmith/Phoenix (Arize AI).

1. Objective

Develop a router-based conversational agent using **LangGraph** or **Llama Index** and **FastAPI**. The bot must handle multi-turn conversations, manage state effectively, perform RAG (Retrieval Augmented Generation), execute function calls with human-in-the-loop validation, and maintain conversation history in MongoDB.

2. System Architecture

The system follows a **Supervisor-Worker (Hub-and-Spoke)** architecture.

- **Entry Point (Supervisor Node):** Receives the user query and classifies the intent. It does *not* answer the question itself; it routes the state to the appropriate specialized node.
- **Worker Nodes:** Specialized sub-graphs or runnables that perform specific tasks and return the result to the conversation flow.

Required Nodes Map

1. **Supervisor Node:** The "Brain" (Router).
 2. **Product Knowledge Node:** DishTV Product Specs (Database Lookup).
 3. **Techchefz Info Node:** RAG Pipeline (Vector Search).
 4. **Recharge Action Node:** Function Calling + Human Approval.
 5. **History Node:** User Transaction History.
 6. **Support Escalation Node:** Handover to human agent.
 7. **Technical Troubleshooting (Bonus):** Step-by-step issue diagnosis.
-

3. Detailed Requirements

A. Data Preparation & Database (MongoDB)

You must set up a MongoDB instance (local or Atlas) with the following collections:

1. **products_collection:**
 - **Task:** Manually scrape/copy text details about "Dish HD", "Dish SMRT Hub", and "Dish Hybrid" from the DishTV website.
 - **Storage:** Store this as structured documents. Each document must have:
 - **product_name:** string
 - **description:** text
 - **source_url:** string (The URL where you got the info)
2. **recharge_history_collection:**
 - Pre-populate with dummy data for a specific **user_id** (e.g., previous recharges, dates, amounts, transaction IDs).
3. **chats_collection:**
 - All interactions must be logged here with a **conversation_id**.

B. The LangGraph Nodes

1. Supervisor Node (The Router)

- **Input:** User's latest message + Conversation History.
- **Function:** Uses an LLM to classify the intent into one of the following:
product_inquiry, **company_info**, **recharge_request**, **history_check**, **escalate_support**, or **general_chat**.
- **Output:** The next node to call.

2. Product Knowledge Node (Database Lookup)

- **Trigger:** Questions like *"What features does Dish SMRT Hub have?"*
- **Logic:**
 1. Extract the product entity from the query.
 2. Query **products_collection** in MongoDB.
 3. Pass the retrieved context to the LLM to generate a natural answer.
- **Requirement:** The final response **must** include the **source_url** (weblink) referenced from the database.

3. Techchefz RAG Node (Vector Search)

- **Trigger:** Questions regarding "Techchefz" (e.g., *"What services does Techchefz offer?"*).
- **Logic (RAG Pipeline):**
 - **Ingestion:** Create a simple text file with information about Techchefz. Chunk it and store embeddings in a vector store (can be MongoDB Vector Search).

- **Query Transformation:** If the user sends multiple messages or a vague follow-up (e.g., *"And where are they located?"*), use an LLM to condense the chat history and new query into a standalone search query *before* vector search.
- **Retrieval:** Fetch the **Top 3** most relevant chunks.
- **Generation:** Synthesize an answer using the retrieved chunks.

4. Recharge Action Node (Function Calling & Pydantic)

- **Trigger:** *"I want to recharge my account."*
- **Logic:**
 1. **Extraction:** Use an LLM with **Pydantic V2 definitions** to extract:
 - `amount` (int)
 - `package_name` (str)
 - `user_id` (str - inferred from context or asked)
 2. **Slot Filling:** If information is missing (e.g., user says *"Recharge my TV"* but doesn't say the amount), the node must loop back and ask for the missing parameter.
 3. **Human-in-the-loop (CRITICAL):** Once all parameters are gathered, the bot **STOP** and return a confirmation request: *"You are about to recharge for [Amount] on [Package]. Do you approve? (Yes/No)"*.
 4. **Execution:** Only if the user says "Yes", execute a mock function `perform_recharge()` and update the `recharge_history_collection`.

5. History Node

- **Trigger:** *"When did I last recharge?"* or *"Show my transaction history."*
- **Logic:** Query `recharge_history_collection` for the current `user_id` and summarize the last 5 transactions in a readable format.

6. Support Escalation Node

- **Trigger:**
 - User explicitly asks *"Talk to human"*.
 - **Sentiment Analysis:** If the user uses negative sentiment words or reports an issue repeatedly (track a loop counter in the state).
- **Logic:** Respond with a standard ticket creation message: *"I am connecting you with a live agent. Your ticket ID is #XYZ."*

4. Technical Constraints & Learning Objectives

A. Pydantic Integration (Mandatory)

- You must use **Pydantic Models** to define the state of your graph and the structure of your structured outputs.

Example:

Python

```
from pydantic import BaseModel, Field
```

```
class RechargeDetails(BaseModel):
    amount: int = Field(..., description="The monetary value of the recharge")
    package: str = Field(..., description="The name of the subscription package")
    user_confirmed: bool = Field(False, description="Has the user explicitly confirmed?")
```

-

B. Conversation Memory

- The system must use `conversation_id`.
- If I ask "Tell me about Dish HD" and then follow up with "How much does it cost?", the bot must know "it" refers to Dish HD.
- Store the state checkpoint in MongoDB (using LangGraph's `MongoDBSaver` checkpointing is a bonus, otherwise manual storage is fine).

C. Observability

- Integrate **LangSmith** or **Phoenix (Arize AI)**.
- Ensure that every LLM call (Supervisor, RAG generator, Extraction) is traced.
- *Submission Requirement:* Provide a screenshot of a trace showing the Supervisor routing to the correct node.

D. FastAPI Implementation

Expose the graph via a REST API.

- `POST /chat`:
 - **Input:** `{"message": "str", "user_id": "str", "conversation_id": "str"}`
 - **Output:** `{"response": "str", "source": "str (optional)", "status": "success"}`

5. Submission Deliverables

1. **Codebase:** A GitHub repository containing:
 - `app.py` (FastAPI + LangGraph logic).
 - `nodes/` (Separate files for Supervisor, RAG, Products, etc.).
 - `models.py` (Pydantic schemas).
 - `requirements.txt`.
2. **Documentation (`README.md`):**
 - How to set up MongoDB and environment variables.
 - Architecture Diagram (Screenshot or Mermaid.js).

6. Evaluation Criteria

- **Architecture:** Is the Supervisor correctly routing intents?
- **Pydantic Usage:** Are models used for validation and structured output?
- **Human-in-the-loop:** Does the recharge node strictly require user approval before "executing"?
- **Data Persistence:** Is history and product data actually coming from MongoDB?
- **Code Quality:** Is the code modular (separate files for nodes) and typed?